

Simulation of a GeoGuessr-inspired Location Selection Algorithm Using OpenStreetMap Data

Giulia Beraldo & Geremia Paoletto

giulia.beraldo@studenti.unipd.it & geremia.paoletto@studenti.unipd.it

Abstract—We study a GeoGuessr-inspired location selection problem on road graphs built from OpenStreetMap data. Starting from an OSM extract of North-East Italy, we construct and simplify a weighted graph, prune non-urban vertices via an unsupervised urbanity score, and detect communities with Leiden clustering. We then compare three sampling strategies, that is round-robin by community, farthest-first traversal, and a hybrid method which for community balance/coverage, spatial diversity, and shortest-path coverage. Results highlight a clear trade-off between community fairness and geometric coverage: round-robin maximizes community spread, while farthest-first yields the best dispersion and covering radius. The full implementation and instructions to reproduce the experiments are available at: <https://github.com/giuliaberaldo2002/LFN>

I. INTRODUCTION

GeoGuessr-style applications ask users to infer a location from street-level imagery, so the quality of the candidate location pool is crucial. If locations are redundant, overly concentrated in a few dense areas, or dominated by uninformative road segments (e.g., long rural stretches), the game becomes less informative and, in the extreme, unnecessarily hard. This motivates selecting a limited budget of locations that are representative of the geographic space while remaining diverse under a meaningful notion of distance. In this work, we study *location selection* from OpenStreetMap (OSM)-derived road data to generate high-quality candidates for downstream tasks such as human-in-the-loop guessing and geographic evaluation.

Despite the practical relevance of this setting, we did not find an openly documented end-to-end implementation of the required sampling pipeline, including the interplay between preprocessing, graph construction, community structure, and diversity-oriented selection. We therefore develop a complete and reproducible approach that can serve as a reference baseline for future work.

Our pipeline consists of four main components:

- **Graph construction and preprocessing:** we build a weighted road graph from OSM and simplify it via degree-2 suppression; we then prune vertices using an unsupervised urbanity score to retain city-like candidates.
- **Community structure:** we partition the candidate graph with the Leiden algorithm optimizing *modularity*, obtaining communities that act as coarse geographic regions for fairness and coverage evaluation.
- **Sampling strategies:** we compare three samplers, that is community round-robin (RR), farthest-first traversal (FFT), and a hybrid approach, across multiple budgets

k , alongside trivial random baselines to isolate preprocessing effects.

- **Evaluation protocol:** we introduce quantitative metrics for community balance/coverage, spatial diversity, urban efficiency, and global coverage under shortest-path distance.

The remainder of the report is organized as follows: Section III defines the problem setting and describes the dataset. Then, Section IV presents preprocessing, community detection, and sampling methods. Section V details the metrics and experimental protocol, and Section VI reports results and Section VII discusses implications and limitations. Finally, Section VIII concludes and outlines future directions.

II. BACKGROUND

We briefly review the concepts needed to describe our pipeline: how OpenStreetMap induces a road graph, how we obtain coarse geographic regions via community detection, and a standard diversity sampler used as a baseline.

A. OpenStreetMap as a road graph

OpenStreetMap (OSM) represents geographic information through *nodes* (points with latitude/longitude) and *ways*, i.e., ordered sequences of nodes forming polylines such as roads, annotated with tags (e.g., *highway*, *maxspeed*). A common abstraction for road-structure tasks is a graph where vertices correspond to OSM nodes and edges connect consecutive nodes along the same way, enabling the use of graph algorithms and shortest-path distances for downstream selection and evaluation.

B. Community detection and the choice of objective

Community detection partitions a graph into vertex groups that are more densely connected internally than externally, by optimizing a quality function. In the final pipeline we use the Leiden algorithm ([1]) optimizing *modularity*, which compares within-community connectivity against a configuration-model null model. Although modularity can exhibit a resolution limit in very large graphs, it typically yields robust, interpretable partitions at a macroscopic scale. In preliminary experiments on our road graphs, CPM (even after tuning its resolution parameter γ) produced overly fine and fragmented partitions, and resulted in uninterpretable plots, which were less suitable as geographic units for our community-based sampling and evaluation. We therefore adopt Leiden modularity as a practical trade-off between interpretability and stability.

C. Diversity sampling via farthest-first traversal

Given a candidate set and a budget k , diversity sampling aims to reduce redundancy by selecting points that are well separated under a chosen distance. *Farthest-first traversal* (FFT) [2] is a simple greedy strategy: starting from a seed, it repeatedly adds the point farthest from the current selected set. For metric distances, FFT provides a 2-approximation for the k -center problem, making it a strong baseline when the goal is dispersion and low covering radius under shortest-path distance.

As a note, in our setting, distances are instantiated as weighted shortest-path distances on the road graph.

III. PROBLEM SETUP AND DATA

We describe the data source, the road-graph construction, and the formal location-selection task.

A. Dataset and road-graph construction

a) Region and source: We use an OpenStreetMap (OSM) road extract for North-East Italy (Veneto, Friuli-Venezia Giulia, Emilia-Romagna, Trentino-Alto Adige) from the Geofabrik distribution.¹ Precisely, this extract contains 69,210,754 nodes and 7,929,412 ways. This region provides substantial geographic diversity (dense cities, rural plains, mountainous terrain) while keeping iteration cycles manageable.

b) Graph construction: From the OSM extract we build an undirected graph $G = (V, E)$ where each OSM node becomes a vertex $v \in V$, and we add an edge $(u, v) \in E$ for every pair of consecutive nodes (u, v) appearing in the same road way. When multiple ways induce the same pair, we merge them into a single edge.

c) Edge weights and distance: Each edge (u, v) is assigned a positive weight $w(u, v)$ equal to the geographic length of the corresponding road segment when available; otherwise we use unit weight. We define $d(\cdot, \cdot)$ as the weighted shortest-path distance induced by w (or hop distance in the unweighted case). This distance is used consistently by distance-based sampling (e.g., FFT) and by our evaluation metrics.

d) Way tags and vertex-level features: OSM semantic attributes (e.g., `highway`, `maxspeed`) are defined at the way level, while our scoring and sampling operate on vertices. We map way-level attributes to vertices by aggregating the attributes of incident edges (and, when needed, of a small neighborhood). The exact feature definitions and aggregation rules are provided in Section IV-B.

B. Operational notion of city-like locations

OSM does not provide a unique ground-truth boundary for cities at the scale of our task. We therefore adopt an *operational* definition: a vertex is considered *city-like* if it lies in a portion of the road network with characteristics of urban fabric, such as high intersection density, short road segments, and a predominance of low-speed/residential road classes. We quantify this notion by computing a vector of local attributes

$x(v)$ for each vertex and an *urbanity score* $\text{score}(v)$ that increases with these indicators. We then restrict the downstream steps to a pruned candidate graph $G_{\text{cand}} = (V_{\text{cand}}, E_{\text{cand}})$ obtained by keeping vertices classified as urban-like (Section IV-C).

C. Task statement

Given the candidate graph $G_{\text{cand}} = (V_{\text{cand}}, E_{\text{cand}})$ and a sampling budget k , the goal is to select a subset

$$S \subseteq V_{\text{cand}}, \quad |S| = k,$$

that balances two competing objectives:

- **structural coverage**, i.e., selecting locations from many different communities (used as coarse geographic regions), and
- **metric diversity**, i.e., selecting locations that are well separated under the road-network distance $d(\cdot, \cdot)$.

These objectives capture the desired behavior of a GeoGuessr-style location pool: avoid redundancy while sampling from multiple distinct urban areas.

IV. METHODS

In this Section we detail the pipeline, from feature computation and pruning to community detection and sampling.

A. Pipeline overview

Our pipeline consists of four stages, that are:

- 1) extraction of OpenStreetMap ways and construction of a road-induced graph (preprocessing);
- 2) computation of node-level features and pruning to retain city-like candidate locations;
- 3) community detection to obtain a structural partition of the graph, ideally divided into different cities;
- 4) budgeted sampling to select k locations according to different objectives;

For clarity, figure 1 summarizes the workflow. Here we added also the evaluation of the three methods for completeness.

B. Graph construction and preprocessing

Starting from the OSM extract, we build a graph $G = (V, E)$ by mapping OSM nodes to vertices and adding an undirected edge between consecutive nodes appearing in the same way. When available, we assign to each vertex its geographic coordinates and a feature vector used by downstream steps, as already described in Section III.

The raw road graph contains many nodes that mainly represent geometric polyline detail rather than semantically meaningful locations. To reduce the computation cost of pruning (see section IV-C), we reduce the size of the initial graph by pruning degree-2 vertices such that if edges $(u, z), (z, v) \in E$, then we prune them substituting with (u, v) whose weight is $w(u, v) = w(u, z) + w(v, z)$. This transformation preserves connectivity and shortest-path distances while substantially reducing graph size, obtaining $G_{\text{simp}} = (V_{\text{simp}}, E_{\text{simp}})$

¹<https://download.geofabrik.de/europe/italy.html>



Fig. 1: Overview of the proposed pipeline and evaluation

C. Features definition

Then, we compute vertex-level features for each $v \in V_{\text{simp}}$ for later use, which are:

- *Node degree*: $\deg(v) = |\mathcal{N}(v)|$: this is due to the observation such that intersections in cities typically have higher degrees (3–4) than rural roads (1–2);
- *Average edge length*: $\bar{l}(v) = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} w(u, v)$: roads in rural areas tend to have longer segments;
- *Road-type tag frequencies extracted from OSM metadata*: with these tags we can infer whether the vertex is in a residential area or in a motorway, which, in the latter case, can be to prune; these attributes are represented respectively as $f_{\text{res}}(v)$ and $f_{\text{mw}}(v)$;
- *Local clustering coefficient* $\text{clust}(v)$: this score measures the density of connections between a node’s neighbours, reflecting the presence of tight, grid-like urban street patterns;
- *Average speed limit*, $\bar{s}(v)$: this represents the value of the tag `maxspeed` (when available).

D. Urbanity score and pruning

After the initial pruning of 2-degree nodes and the computation of the node-level features, the graph still contains vertices belonging to non-city-like regions (e.g., highways, rural connectors). To retain a set of candidates concentrated on urban areas, we compute an *urbanity score* for each vertex, basing its computation on node-level features (see section IV-C) and prune the graph accordingly. In the following paragraphs we give more details about this procedure.

a) Feature transforms and standardization: Before scoring, we apply stable transforms: we use $\log(1 + \deg(v))$ for degree, and we winsorize heavy-tailed continuous features by capping the upper tail at the p -th percentile (default $p = 99.5$) for $\bar{l}(v)$ and (when used) $\bar{s}(v)$. We then z-score each feature via a standard scaler, obtaining a standardized feature vector $z(v) \in \mathbb{R}^d$ (with $d = 5$ without `maxspeed` and $d = 6$ with `maxspeed`).

b) Urbanity score definition: Given the standardized feature vector, we define the urbanity score as a linear model:

$$\text{score}(v) = w^\top z(v),$$

where $w \in \mathbb{R}^d$ is a weight vector and larger scores are intended to correspond to more urban-like vertices (higher degree/residential frequency/clustering, shorter segments, lower motorway frequency, and optionally lower speed limits).

c) Manual tuning via constrained random search:

Rather than fitting w via clustering, we tune it using a constrained random search with sign constraints reflecting domain knowledge. We run a random search over $T = 800$ candidate weight vectors, evaluated on a uniform random subset of at most $n = 80,000$ vertices (seed = 0). Each candidate weight is sampled independently from feature-specific ranges that fix the sign, as follows: $w_{\log \deg} \sim \mathcal{U}(0.6, 1.8)$, $w_{\text{clust}} \sim \mathcal{U}(0.1, 1.0)$, $w_{\text{freq}_{\text{res}}} \sim \mathcal{U}(0.2, 1.6)$, $w_{\bar{l}} \sim \mathcal{U}(-1.8, -0.2)$, $w_{\text{freq}_{\text{mw}}} \sim \mathcal{U}(-1.6, -0.1)$, $w_{\bar{s}} \sim \mathcal{U}(-1.8, -0.2)$ (if enabled). Each sampled vector is normalized to unit norm, $w \leftarrow w / \|w\|_2$, for numerical stability in z-scored space.

d) Tuning objective (quantile separation): For a candidate w , let $s(v) = w^\top z(v)$ be the score on the tuning sample. Fix $q = 0.2$ and define the top and bottom quantile sets:

$$S_q = \{v : s(v) \leq Q_q\}, \quad S_{1-q} = \{v : s(v) \geq Q_{1-q}\},$$

where Q_q is the q -quantile of the score distribution. For each standardized feature f , define a separation term

$$\Delta_f = \mathbb{E}[f \mid S_{1-q}] - \mathbb{E}[f \mid S_q].$$

We maximize the weighted separation objective $J(w) = 1.00 \Delta_{\log \deg} + 0.80 \Delta_{\text{freq}_{\text{res}}} + 0.25 \Delta_{\text{clust}} - 0.80 \Delta_{\bar{l}} - 0.60 \Delta_{\text{freq}_{\text{mw}}} - 0.50 \Delta_{\bar{s}}$, (with the last term omitted when `maxspeed` is disabled). To prevent degenerate solutions, we additionally apply hard penalties when any key separation has the wrong sign (e.g., $\Delta_{\log \deg} < 0$ or $\Delta_{\bar{l}} > 0$). We finally select $w^* = \arg \max_w J(w)$ over the T trials.

e) GMM-based pruning rule: After computing $\text{score}(v)$ for all vertices, we fit a 2-component Gaussian Mixture Model (GMM) on the 1D score distribution (random state = 0). We designate as *urban* the component with the larger mean score, and keep vertices whose posterior probability of belonging to the urban component exceeds a threshold τ (default $\tau = 0.5$):

$$V_{\text{cand}} = \{v \in V : \Pr(\text{urban} \mid \text{score}(v)) \geq \tau\}.$$

The induced subgraph $G_{\text{cand}} = (V_{\text{cand}}, E_{\text{cand}})$ is then passed to community detection and sampling.

f) Feature vector: For each vertex $v \in V$, we compute a feature vector

$$x(v) = [\log(1 + \deg(v)), \text{clust}(v), f_{\text{res}}(v), \bar{l}(v), f_{\text{mw}}(v), \bar{s}(v)],$$

The resulting spatial distribution of the candidate graph is illustrated in Figure 2. As shown, the pruning process effectively isolates dense urban clusters while discarding sparse rural connections, naturally fragmenting the network into distinct disconnected components.



Fig. 2: Visualization of the pruned road network.

g) *Handling disconnected components (virtual edges):*

After urban pruning, the candidate graph may become disconnected. Since several of our sampling strategies and metrics rely on shortest-path distances, we enforce connectivity by adding a small set of *virtual* edges between components. Concretely, we compute the connected components of the pruned graph and select one *representative* vertex per component as the vertex closest to the geometric centroid of that component (using vertex coordinates). We then build a sparse k -nearest-neighbor graph (with $k = 10$) over the component representatives using a KD-tree, and compute its minimum spanning tree (MST). Each MST edge induces a virtual edge between the corresponding representative vertices in the original graph, with weight equal to their Euclidean distance (scaled to meters when coordinates are in latitude/longitude) multiplied by a constant tortuosity factor. If the k -NN graph is not fully connected, we additionally connect the resulting MST forest by computing an MST over the forest components. All added edges are labeled with `type=virtual` and are used only to restore reachability for distance-based computations. Restoring connectivity via virtual edges enables global shortest-path distances, but it introduces an approximation that may impact distance-based sampling and metrics; however, a more thorough sensitivity analysis is left as future work.

E. Community detection

To capture large-scale structure (e.g., different urban areas or neighborhoods), we partition the pruned candidate graph G_{cand} into communities. We use the Leiden algorithm due to its refinement phase that tends to avoid poorly connected communities, which is important when communities are used downstream as units for sampling and analysis.

a) *Objective function (final choice):* In the final pipeline we optimize *modularity*. Although modularity can merge small clusters in very large graphs (resolution limit), we found it to provide communities at a more appropriate granularity for our downstream goal: using communities as proxies for city-like

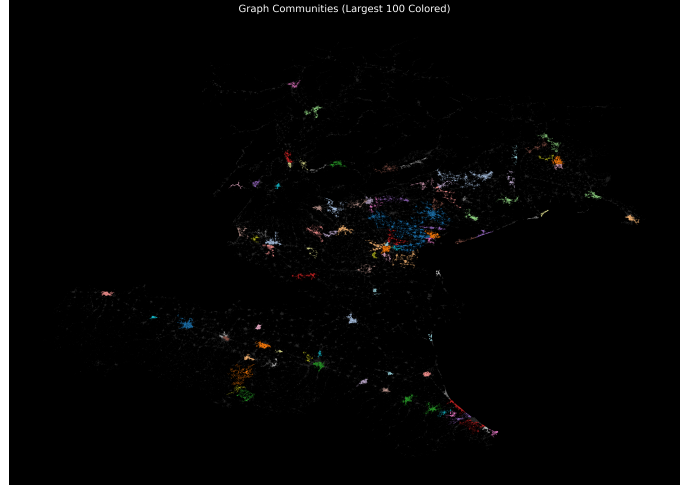


Fig. 3: Visualization of the 100 biggest communities detected by the Leiden algorithm.

regions in fairness and coverage metrics. In contrast, CPM (even after tuning γ) often produced partitions that were too fine and fragmented on our road graphs, resulting in many small components that were less meaningful for sampling. Therefore, modularity yields a better trade-off between interpretability and usefulness for our sampling objectives. The resulting partition is shown in Figure 3, which illustrates how the algorithm successfully groups spatially contiguous nodes into distinct, semantically meaningful urban zones.

F. Sampling methods

Given the pruned road graph $G_{\text{cand}} = (V_{\text{cand}}, E_{\text{cand}})$, we aim to select a subset of k locations $\mathcal{S} \subset V_{\text{cand}}$ ($|\mathcal{S}| = k$). As we have already observed, the selection process is governed by obtaining optimal values of *coverage* (representing different communities) and *diversity* (maximizing spatial separation).

As we already defined, we define the distance $d(u, v)$ between two vertices, $u, v \in V_{\text{cand}}$, as the shortest-path distance on the graph G_{cand} .

a) *Trivial baselines (Dummy1/Dummy2):* To isolate the effect of preprocessing and to contextualize the benefit of our structured samplers, we include two simple random baselines: Dummy1 samples k vertices uniformly at random from the original road graph G (before any pruning). Dummy2 samples k vertices uniformly at random from the simplified graph G_{simp} obtained after degree-2 suppression, but *without* applying the urbanity (GMM) pruning. These baselines introduce no diversity or community-aware logic and therefore act as lower bounds, allowing us to quantify how much improvement comes purely from preprocessing versus from the sampling strategies.

1) *Algorithms:* We therefore implement three different sampling algorithms, which sample $k \in \mathbb{Z}^+$ vertices such that the objectives of maximizing diversity and coverage are addressed.

a) *Method 1: Community Round Robin (RR)*: Instead of allocating static quotas proportional to community sizes (which biases selection towards large, dense areas), we adopt a cyclic approach to enforce maximum fairness across regions detected by the Leiden algorithm. Let $\mathcal{C} = \{C_1, \dots, C_m\}$ be the set of communities. The algorithm iterates through the communities in a randomized order and selects one node $v \in C_i$ without replacement at each step.

- **Procedure:** In cycle t , we select a node s_t from $C_{(t \bmod m)}$. Communities with no remaining vertices are skipped. In the unlikely event that all communities are exhausted before reaching the budget k , we stop early.
- **Intuition:** This strictly enforces that small, peripheral communities are represented as early as large central ones.
- **Computational Complexity** We precompute the vertex lists for each community in $O(|V|)$ time. Sampling then takes $O(k)$ time (constant-time selection per sample by maintaining a shuffled list and an index pointer per community), with $O(r)$ additional overhead to maintain/skip exhausted communities (where $r = |\mathcal{C}|$).

b) *Method 2: Graph-based Farthest-First Traversal (FFT)*: This method adapts the heuristic for the k -center problem to the graph metric space. It greedily maximizes the minimum distance from the current set S . Starting from an initial seed s_1 (e.g., the node with maximum degree), at each step $t + 1$, we select:

$$s_{t+1} = \arg \max_{v \in V_{\text{cand}} \setminus S_t} \left(\min_{u \in S_t} d(v, u) \right) \quad (1)$$

- **Computational Complexity:** FFT performs $k - 1$ iterations; at each iteration it needs the distances from the newly selected center to all vertices. On a weighted graph, using Dijkstra's algorithm with a binary heap, one single-source shortest-path computation costs $O((|E| + |V|) \log |V|)$. Therefore, the overall time complexity is

$$O(k(|E| + |V|) \log |V|),$$

with $O(|V|)$ additional memory to store the current nearest-center distances.

- **Intuition:** FFT maximizes the explicit spatial diversity (separation) and minimizes the maximum radius $R = \max_v d(v, S)$, ensuring global coverage even without knowledge of community structures.

c) *Method 3: Hybrid City-Ordered Round Robin*: To combine the structural awareness of communities with the spatial spread of FFT, we introduce a hybrid approach.

- 1) **Macro-ordering:** As a first step, we identify a representative node r_i for each community C_i (e.g., $\arg \max_{v \in C_i} \deg(v)$), under the assumption that the number of communities m is larger than the number of samples k . We then run FFT on the set of representatives to select k communities $\mathcal{O} = (C_{\pi(1)}, C_{\pi(2)}, \dots, C_{\pi(k)})$.
- 2) **Micro-sampling:** We perform Round Robin sampling on the selected communities.

This strategy aims to sample from spatially distant communities first, mitigating the risk of Round Robin selecting from adjacent (and thus redundant) communities in early iterations.

V. EVALUATION

A. Evaluation Goals

To assess the quality of a sampled set S of size k , we consider three partially conflicting objectives:

- **Structural fairness (community coverage and balance).** The sampler should not neglect peripheral or small regions: selected nodes should cover many Leiden communities and distribute the budget reasonably evenly across them, rather than concentrating on a few large/dense areas.
- **Global coverage (spatial completeness).** The sampled set should act as a skeleton of the candidate graph, keeping every vertex close to at least one selected point. This corresponds to the metric k -center objective.
- **Spatial diversity.** Selected points should be well separated under the chosen distance, reducing redundancy and increasing informativeness of the location pool.

B. Metrics

We quantify the above objectives using the following metrics (all distances use shortest-path distance on the weighted road graph).

a) *Community coverage and balance.*: Let the Leiden partition be $\mathcal{C} = \{C_1, \dots, C_m\}$ and let $n_i = |C_i \cap S|$ be the number of samples in community i . We report *coverage fraction*

$$\text{Cov} = \frac{|\{i \in [m] : n_i > 0\}|}{m},$$

and two balance measures over the covered communities $\mathcal{I} = \{i : n_i > 0\}$. First, normalized entropy:

$$H_{\text{norm}} = \frac{-\sum_{i \in \mathcal{I}} p_i \ln p_i}{\ln(\min(k, |\mathcal{I}|))}, \quad p_i = \frac{n_i}{k}. \quad (2)$$

Second, the coefficient of variation (CV) of the nonzero counts:

$$\text{CV} = \frac{\text{std}(\{n_i\}_{i \in \mathcal{I}})}{\text{mean}(\{n_i\}_{i \in \mathcal{I}})}.$$

High entropy / low CV indicate a more balanced allocation across the communities that are actually represented.

b) *Global coverage radius (R_{max})*: For each vertex $v \in V_{\text{cand}}$, let $\delta(v) = \min_{s \in S} d(v, s)$ be its distance to the nearest sample. We report the worst-case covering radius

$$R_{\text{max}} = \max_{v \in V_{\text{cand}}} \delta(v), \quad (3)$$

and the 90th percentile p_{90} of $\{\delta(v)\}_{v \in V_{\text{cand}}}$ as a robust summary less sensitive to extreme peripheral vertices.

c) *Pairwise diversity.*: We measure dispersion via pairwise shortest-path distances between distinct samples:

$$\text{Div}_{\min} = \min_{s_i \neq s_j \in S} d(s_i, s_j),$$

and we also report a lower-tail summary such as the 10th percentile of pairwise distances (p_{10}), to capture typical separation rather than only the single closest pair.

d) Clustering agreement (city-likeness proxy).: We induce a Voronoi-style partition of V_{cand} by assigning each vertex v to its nearest sample $s \in \mathcal{S}$ (ties broken arbitrarily), and compare the induced clusters to Leiden communities using Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). Higher agreement indicates that the sampled set preserves community-level structure.

VI. RESULTS

In all experiments, we fix the sampling budget to $k = 500$ selected vertices. For plotting the graphs we used $k' \in \{10, 20, 50, 100, 200, 500\}$ and for each k' we average over 4 seeds. Since the Leiden partition may contain a very large number of communities (including many very small ones), we restrict our analysis to the 500 largest communities (since they cover approximately 80% of the vertices) and discard the remaining ones from the evaluation. Unless otherwise stated, community-based metrics (e.g., coverage and balance) are computed with respect to this subset of 500 largest communities.

A. Experimental results

We evaluate the sampling strategies along three axes:

- **community fairness** (balance and coverage across Leiden communities);
- **spatial diversity** (pairwise separation in shortest-path distance);
- **global coverage** (how well the selected set covers the candidate graph in terms of distance-to-nearest-sample).

a) Community fairness: RR/Hybrid are near-perfect by design.: Figures 4 and 5 show that round-robin attains essentially perfect balance across covered communities: the coefficient of variation (CV) of per-community sample counts remains near 0, and normalized entropy remains near 1, across all budgets k . This is expected since RR allocates samples cyclically across communities rather than proportionally to community size. The hybrid method exhibits the same behavior in our configuration because it first selects a subset of communities and then applies the same round-robin allocation within that set (in particular, when $m = k$ it assigns one sample per selected community). In contrast, FFT shows noticeably worse balance (higher CV and lower entropy), indicating that a purely distance-driven selection naturally concentrates samples in a limited subset of communities when community structure and geometric spread do not align perfectly.

b) Community coverage: RR/Hybrid maximize breadth.: Figure 6 confirms that RR and Hybrid maximize the number of distinct communities touched by the sample set at fixed budget k . Random baselines (Dummy1 and Dummy2) increase coverage more slowly because repeated random draws tend to fall in already-covered large communities. FFT tends to cover fewer communities than RR/Hybrid since its objective optimizes dispersion and covering radius rather than explicit community breadth.

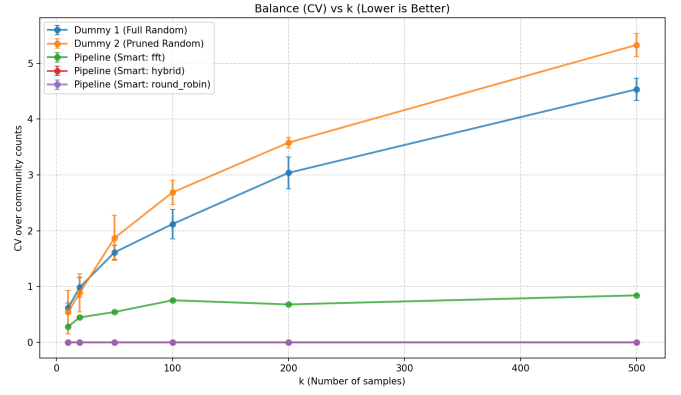


Fig. 4: Coefficient of Variation (CV) of per-community sample counts versus budget k . Lower values indicate better balance.

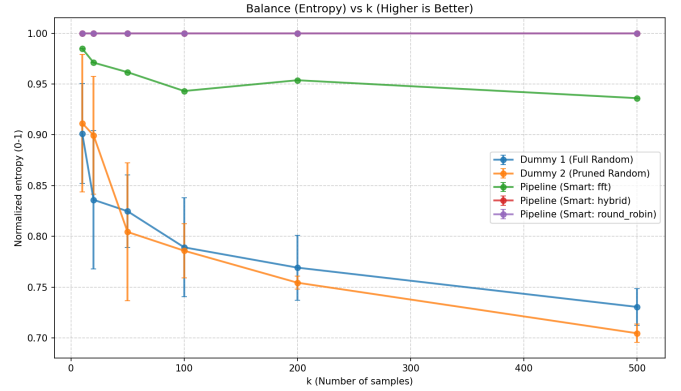


Fig. 5: Normalized entropy of the sample distribution across communities. Values closer to 1 imply a more uniform allocation.

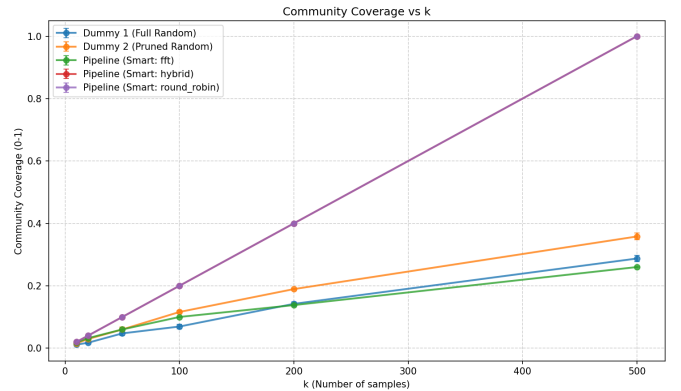


Fig. 6: Community coverage analysis. Number of distinct Leiden communities covered by the selected locations as the sampling budget k increases.

c) Spatial diversity: FFT dominates pairwise separation.: Figures 7 and 8 show that FFT strongly outperforms the other strategies in terms of pairwise separation: both the 10th percentile of pairwise distances and the minimum separation are substantially larger, especially at small k . This matches the

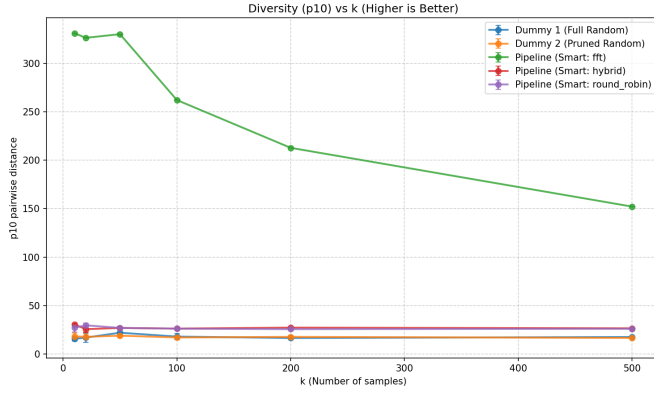


Fig. 7: Spatial diversity, 10th percentile of pairwise shortest-path distances. Higher values indicate that even relatively close pairs of selected nodes maintain good separation.

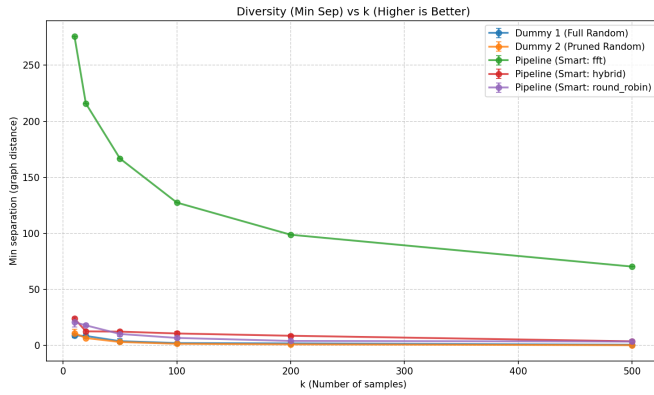


Fig. 8: Minimum pairwise separation distance. Distance between the two closest selected nodes.

theoretical behavior of farthest-first traversal, which greedily chooses each new sample as far as possible from the current set, thereby avoiding redundancy by construction. In contrast, RR/Hybrid and the random baselines do not explicitly optimize separation and can select multiple locations within the same dense area, causing the lower tail (and worst-case) separations to drop as k increases.

d) Urban efficiency: isolating the benefit of pruning.:

Figure 9 shows that all pipeline samplers (RR/Hybrid/FFT) achieve urban efficiency equal to 1 because they sample from the urban candidate set V_{cand} by construction. The Dummy1 baseline (random sampling on the unpruned graph) selects a non-trivial fraction of non-urban vertices, confirming that an explicit urban-filtering stage is necessary to reliably focus on city-like points. The Dummy2 baseline (random sampling after degree-2 suppression only) partially mitigates this effect by biasing selection toward structurally salient vertices (e.g., intersections), but still lacks an explicit notion of city-likeness.

e) Global coverage: FFT yields the best covering radius.:

Figures 10 and 11 show that FFT achieves the lowest distance-to-nearest-sample statistics as k increases, both for the robust p_{90} and for the worst-case radius R_{max} . This is expected since

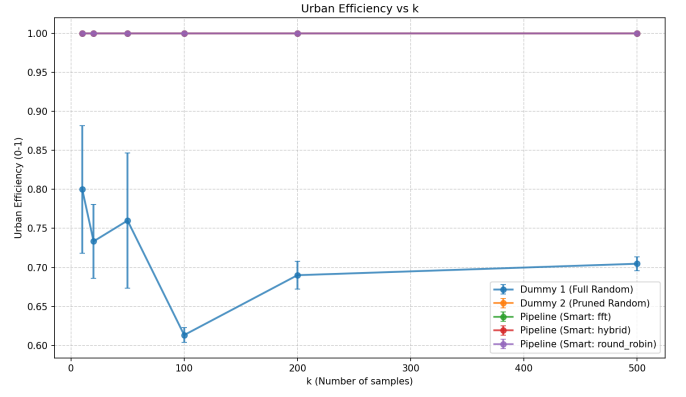


Fig. 9: Urban efficiency comparison. Fraction of sampled nodes that satisfy the urbanity threshold.

FFT is a greedy heuristic for the metric k -center problem, directly targeting the covering radius objective. Fairness-oriented samplers (RR and Hybrid) exhibit substantially larger p_{90} and R_{max} , reflecting an inherent trade-off: enforcing balanced community representation can prevent placing samples at globally optimal “centers” that minimize the covering radius. Overall, the results highlight a consistent tension between fairness across communities, dispersion among selected locations and global coverage quality of the candidate graph.

f) Code availability and reproducibility.: All code developed for this project, together with usage instructions to reproduce the preprocessing pipeline, run the sampling baselines and proposed methods, and regenerate the experimental results (tables/figures), is publicly available at <https://github.com/giulialberaldo2002/LFN>.

VII. DISCUSSION

Our experiments reveal a consistent set of trade-offs between structural fairness, spatial diversity, and global coverage. These trade-offs are not artifacts of a particular metric, but follow directly from the objectives optimized (explicitly or implicitly) by each sampling strategy.

a) Why RR/Hybrid are fair but not coverage-optimal:

Round-robin enforces balanced allocation across communities by construction: each community receives samples early, preventing domination by large or dense regions. This explains its near-perfect balance and strong community coverage. However, fairness alone does not guarantee that selected locations are good global “centers” of the graph. When communities are geographically extended or irregular, enforcing one sample per community can place points in suboptimal positions for minimizing distances, which in turn yields larger p_{90} and R_{max} compared to distance-driven methods.

b) Why FFT excels at diversity and global coverage:

Farthest-first traversal is a greedy heuristic for the metric k -center problem, and therefore directly targets the covering-radius objective. As a consequence, it produces small p_{90} and R_{max} and maintains strong pairwise separation. The downside is that the resulting set is not constrained to represent many

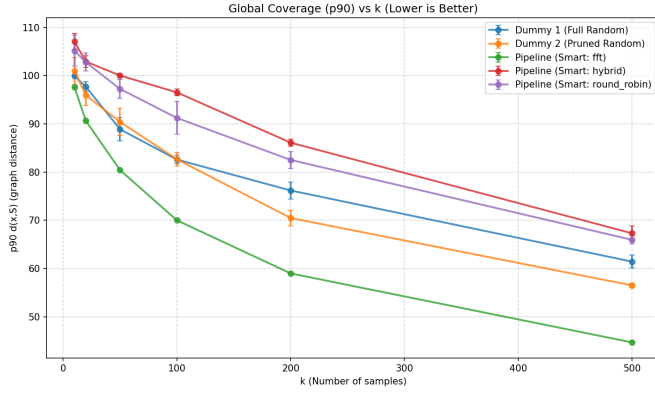


Fig. 10: Global coverage performance. 90th percentile of the shortest-path distance from any network node to its nearest selected sample. Lower is better.

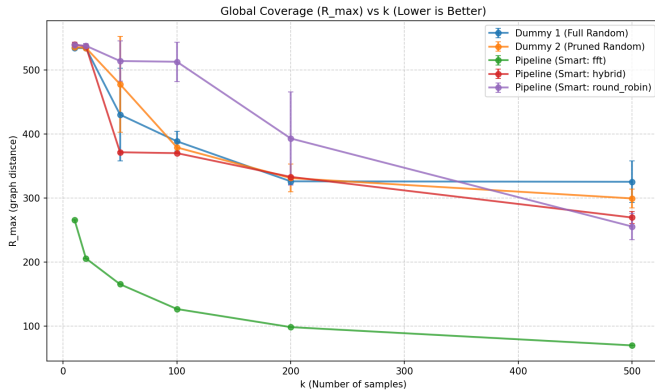


Fig. 11: Worst-case covering radius. Maximum distance a user would have to travel to reach a sampled location.

communities: if multiple communities are close in graph distance, FFT may select multiple points within the same structural region while leaving other communities uncovered, explaining its weaker balance/coverage metrics.

c) Role of preprocessing and the random baselines:

The dummy baselines help isolate the contribution of preprocessing. Random sampling on the raw graph tends to pick uninformative points, since the vertex distribution is dominated by non-urban road segments. Degree-2 suppression partly alleviates this by collapsing long chains into weighted edges, increasing the relative prominence of intersections. Nonetheless, explicit urbanity pruning is necessary to reliably restrict the pool to city-like candidates, as reflected by the urban efficiency results.

d) *Practical implications:* The choice of sampler should be guided by the downstream application. If the goal is to provide a uniformly representative pool across many geographic subregions (e.g., to ensure variety for human guessing), RR/Hybrid are appropriate. If the goal is to ensure that every part of the graph is close to a selected point (e.g., for coverage-driven evaluation or landmark placement), FFT provides a strong baseline. Hybrid approaches can interpolate

between these objectives by restricting FFT to a subset of communities or by allocating a portion of the budget to each criterion.

VIII. CONCLUSION AND FUTURE WORK

In this report we studied a GeoGuessr-inspired *location selection* problem on road networks derived from OpenStreetMap. We presented an end-to-end and reproducible pipeline that constructs a weighted road graph from OSM data, simplifies it via degree-2 suppression, restricts the candidate pool through an urbanity-driven pruning stage, extracts coarse geographic structure via Leiden community detection, and selects a budgeted set of k locations using sampling strategies that trade off fairness and geometric coverage. To evaluate these strategies, we introduced a set of complementary metrics capturing community coverage/balance, spatial diversity, and global coverage under shortest-path distance, and we included simple random baselines to isolate the contribution of preprocessing.

Empirically, the results reveal a clear and consistent trade-off. Community-aware strategies such as Round-Robin (and the Hybrid configuration used here) maximize fairness and breadth, covering many communities with near-uniform allocation. In contrast, Farthest-First Traversal achieves the strongest spatial dispersion and the best global coverage (smallest p_{90} and $R_{\max}$), behaving as an effective k -center heuristic on the road graph. These findings suggest that the “best” sampler depends on the downstream objective: fairness-oriented selection is preferable when geographic variety across regions is required, while FFT is preferable when worst-case or robust coverage is the primary goal.

a) *Future work:* There are several concrete directions to strengthen and extend this pipeline:

- **Choice of CPM:** While CPM offers explicit control via γ , however in our experiments it fragmented; future work could revisit CPM and merging techniques.
- **Automated community-resolution selection:** replace manual tuning with an explicit criterion (e.g., stability across repeated Leiden runs, target community-size constraints, or plateau detection in quality/stability curves);
- **Scaling and robustness:** evaluate on larger and more diverse geographic regions and report sensitivity to pruning thresholds and graph simplification parameters;
- **Stronger baselines and hybrids:** include additional structured samplers (e.g., stratified sampling proportional to community size, multi-stage FFT with per-community quotas, or budget-splitting schemes that allocate a fraction of samples to fairness and the rest to k -center optimization) to better map the Pareto frontier between fairness and coverage.

REFERENCES

- [1] V. A. Traag, L. Waltman, and N. J. Van Eck, “From louvain to leiden: guaranteeing well-connected communities,” *Scientific reports*, vol. 9, no. 1, pp. 1–12, 2019.
- [2] T. F. Gonzalez, “Clustering to minimize the maximum intercluster distance,” *Theoretical computer science*, vol. 38, pp. 293–306, 1985.

IX. CONTRIBUTION OF EACH MEMBER

The contributions detailed in this section pertain to the development phase spanning from the midterm deliverable to the final project submission. For a comprehensive overview of the preliminary activities and initial network analysis, please refer to the previously submitted midterm report.

- **Giulia Beraldo:**

- Developed the evaluation framework of the sampling algorithms, implementing code for computing performance metrics.
- Designed and implemented the novel hybrid sampling strategy.
- Completed and edited most of the final manuscript.

- **Geremia Paoletto:**

- Developed visualization pipelines for graph pruning processes and clustering results.
- Implemented algorithmic solutions to restore connectivity among disconnected graph components post-pruning.
- Optimized the code for scalability and deployment on the cluster.
- Executed the pipeline and simulations on the cluster, gathering final results.
- Tuned the Leiden algorithm implementation, specifically targeting Modularity maximization.

Task (Post-Midterm)	G. Beraldo	G. Paoletto
Evaluation framework & Metrics implementation	Lead	–
Hybrid sampling strategy (Design & Code)	Lead	–
Pruning & Clustering visualization	–	Lead
Connectivity restoration logic	–	Lead
HPC optimization & Simulation runs	–	Lead
Leiden refinement (Modularity max.)	Support	Lead
Final technical report writing	Lead	Support
Estimated fraction of work	50%	50%

Detailed contribution summary for the final project phase.

X. AI USAGE POLICY

Generative AI tools were utilized to support specific auxiliary aspects of the project. ChatGPT was employed for linguistic refinement, grammar correction, and assisting with L^AT_EX syntactic structures.

Gemini was used for conceptual brainstorming and methodological validation, specifically regarding:

- The formal definition and interpretation of network evaluation metrics;
- Strategies for the effective visualization of performance results;
- Algorithmic optimization and scalability strategies for handling large-scale network data.

Disclaimer: All final technical decisions, code implementations, and reported results were manually verified and finalized by the authors.